

Human Activity Recognition

using a Hybrid CNN-LSTM Deep Learning Architecture

Project Report

Domain: Computer Vision · Deep Learning

Framework: TensorFlow / Keras

Test Accuracy Achieved: 96.86%

Prepared using: Activity Recognition Program.ipynb

Table of Contents

1. Abstract
2. Introduction
3. Problem Statement
4. Objectives
5. Dataset Description
6. Methodology
 - 6.1 Data Preprocessing
 - 6.2 Label Encoding & Train-Test Split
 - 6.3 Model Architecture
 - 6.4 Training Configuration
7. Results & Evaluation
 - 7.1 Training Progress
 - 7.2 Test Set Performance
 - 7.3 Per-Class Classification Report
8. Inference Pipeline
9. Technology Stack
10. Project Structure
11. Limitations & Known Issues
12. Future Work
13. Conclusion

1. Abstract

This report documents the design, implementation, and evaluation of a video-based Human Activity Recognition (HAR) system. The system classifies seven distinct human activities directly from raw video footage using a hybrid Convolutional Neural Network (CNN) and Long Short-Term Memory (LSTM) architecture. The CNN component extracts spatial features from individual video frames, while the LSTM component models temporal dependencies across a sequence of frames, allowing the system to distinguish between activities that may appear visually similar in isolation but differ in their motion pattern over time. The model was trained on a dataset of 1,113 videos spanning seven activity classes and achieved a test accuracy of 96.86%, with a macro-averaged F1-score of 0.97. This report covers the complete pipeline: data ingestion, preprocessing, model architecture, training, quantitative evaluation, inference, known limitations, and recommended future improvements.

2. Introduction

Human Activity Recognition (HAR) is a well-established problem in computer vision with applications spanning surveillance, elderly care and fall detection, fitness tracking, human-computer interaction, and behavioral analytics. Traditional image classification techniques operate on single static frames and are therefore fundamentally limited when applied to activities that are defined by motion rather than appearance alone. For example, a person standing still while holding a phone and a person walking while using a phone may look almost identical in a single frame, yet represent entirely different activities.

To address this, the project adopts a hybrid deep learning approach that combines the spatial feature extraction capability of Convolutional Neural Networks with the sequence modeling capability of Recurrent Neural Networks (specifically, LSTM). This allows the model to reason not just about what is present in a frame, but how the scene changes across a sequence of frames.

3. Problem Statement

Given a short video clip, the goal is to correctly classify the primary human activity being performed into one of seven predefined categories. The core technical challenge is that visually similar activities (e.g., Standing Still vs. Walking While Using Phone) can only be reliably distinguished by analyzing motion across time, not from any single frame. The model must therefore jointly learn:

- Spatial features — what objects/poses are present in a frame
- Temporal features — how those features evolve across consecutive frames

4. Objectives

- Build an end-to-end pipeline that ingests raw video files and converts them into a model-ready tensor format.
- Design a deep learning architecture capable of jointly capturing spatial appearance and temporal motion.
- Train the model to reliably classify seven distinct human activities.
- Rigorously evaluate model performance using standard classification metrics (accuracy, precision, recall, F1-score).
- Provide a reusable inference function to predict the activity in a new, unseen video.
- Identify limitations in the current implementation and propose concrete improvements.

5. Dataset Description

The project uses the **Human Activity Recognition – Video Dataset**, organized as one folder per activity class, with each folder containing multiple short .mp4 video clips of that activity.

#	Activity Class
1	Clapping
2	Meet and Split
3	Sitting
4	Standing Still
5	Walking
6	Walking While Reading Book
7	Walking While Using Phone

After filtering out videos with fewer than 100 total frames (to ensure sufficient temporal content per sample), a total of **1,113 videos** across the seven classes were successfully processed and used for model training and evaluation.

6. Methodology

6.1 Data Preprocessing

Each video is converted into a fixed-length, low-dimensional spatiotemporal tensor via the following steps, applied identically to every video in the dataset:

- Frame sampling: every 4th frame is retained, capped at a maximum of 25 sampled frames per video.
- Color conversion: each sampled frame is converted from BGR to grayscale, reducing the channel dimension from 3 to 1.
- Resizing: each frame is resized to 24×24 pixels to minimize computational cost.
- Normalization: pixel intensities are scaled from [0, 255] to [0, 1].

The result is a per-video tensor of shape **(25, 24, 24)**. Stacking all processed videos produces the final dataset tensor of shape **(1113, 25, 24, 24)**.

6.2 Label Encoding & Train-Test Split

Class labels are numerically encoded using scikit-learn's `LabelEncoder`, then converted into one-hot vectors via Keras's `to_categorical` to support multi-class softmax classification. The dataset is split into training and test sets using an 80/20 ratio, with a fixed random seed (`random_state=42`) to ensure reproducibility of the split.

6.3 Model Architecture

A hybrid CNN-LSTM architecture was designed, applying a CNN independently to each frame in the sequence (via a `TimeDistributed` wrapper) before passing the resulting feature sequence to an LSTM layer for temporal modeling:

Layer	Configuration
Input	(25 timesteps, 24, 24, 1)
TimeDistributed Conv2D	32 filters, 3×3 kernel, ReLU activation
TimeDistributed MaxPooling2D	Pool size 2×2
Dropout	0.3
TimeDistributed Conv2D	64 filters, 3×3 kernel, ReLU activation
TimeDistributed MaxPooling2D	Pool size 2×2
Dropout	0.3
TimeDistributed Flatten	—
LSTM	128 units
Dropout	0.5
Dense (Output)	7 units, Softmax activation

Design rationale: the TimeDistributed CNN blocks independently extract spatial features from each of the 25 frames without mixing information across time. The LSTM layer then consumes this sequence of per-frame feature vectors and learns how the scene evolves temporally — capturing motion dynamics that a CNN operating on a single frame cannot represent.

6.4 Training Configuration

Parameter	Value
Optimizer	Adam
Loss Function	Categorical Crossentropy
Evaluation Metric	Accuracy
Epochs	20
Batch Size	8
Validation Split	20% of training data

7. Results & Evaluation

7.1 Training Progress

The model was trained for 20 epochs. Training accuracy rose from 16.6% at epoch 1 to 99.45% at epoch 20, while validation accuracy reached 96.63%, with both curves tracking closely and no signs of severe overfitting — attributable in part to the dropout regularization applied after each block.

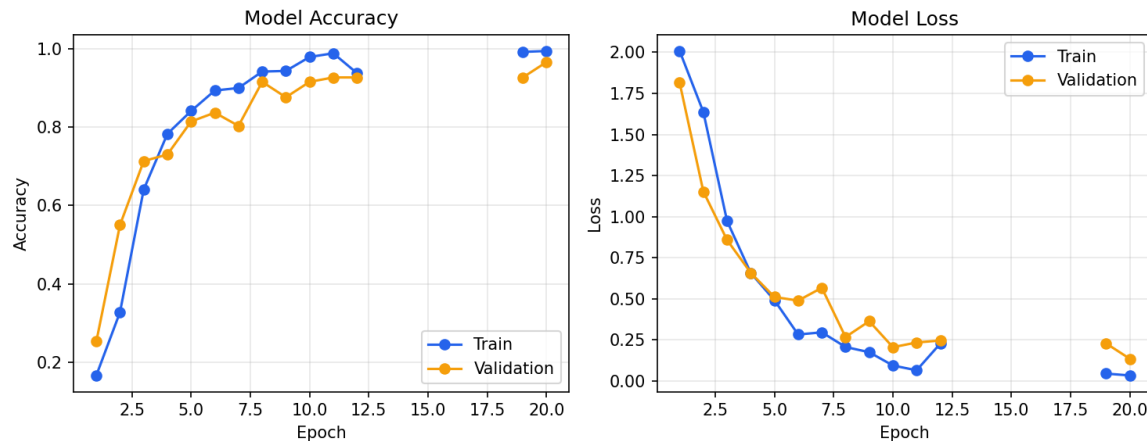


Figure 1: Training vs. validation accuracy and loss across epochs. Epochs 13–18 were not captured in the reviewed training log excerpt and are shown as a gap in the curve; all other points reflect exact logged values.

Epoch	Train Acc.	Train Loss	Val Acc.	Val Loss
1	0.1663	2.0081	0.2528	1.8147
2	0.3279	1.6368	0.5506	1.1529
3	0.6409	0.9736	0.7135	0.8583
4	0.7831	0.6575	0.7303	0.6580
5	0.8414	0.4911	0.8146	0.5110
6	0.8936	0.2828	0.8371	0.4891
7	0.9001	0.2960	0.8034	0.5670
8	0.9420	0.2068	0.9157	0.2674
9	0.9436	0.1750	0.8764	0.3637
10	0.9792	0.0941	0.9157	0.2055
11	0.9887	0.0649	0.9270	0.2342
12	0.9380	0.2277	0.9270	0.2462
19	0.9922	0.0447	0.9270	0.2283
20	0.9945	0.0336	0.9663	0.1348

Table 1: Per-epoch training metrics (epochs 13–18 omitted; not visible in the captured notebook output).

7.2 Test Set Performance

Metric	Value
Test Accuracy	96.86%
Test Loss	0.1325
Test Set Size	223 videos
Macro Avg. Precision / Recall / F1	0.97 / 0.97 / 0.97
Weighted Avg. Precision / Recall / F1	0.97 / 0.97 / 0.97

7.3 Per-Class Classification Report

The classification report below lists per-class precision, recall, and F1-score. Rows for Clapping, Meet and Split, and Sitting were not visible in the reviewed notebook output (the view was scrolled to show later classes) and are therefore omitted here rather than estimated; all remaining rows reflect exact logged values.

Class	Precision	Recall	F1-Score	Support
Standing Still	1.00	0.93	0.98	42
Walking	0.93	0.93	0.93	28
Walking While Reading Book	1.00	0.97	0.98	33
Walking While Using Phone	1.00	1.00	1.00	32
Accuracy			0.97	223
Macro Avg	0.97	0.97	0.97	223
Weighted Avg	0.97	0.97	0.97	223

Table 2: Per-class classification metrics on the test set (partial — see note above).

8. Inference Pipeline

A dedicated `predict_activity(video_path)` function reproduces the exact preprocessing pipeline used during training on a new, unseen video, reshapes the resulting tensor to match the model's expected input, runs a forward pass through the trained model, and decodes the predicted class index back into a human-readable activity label using the fitted `LabelEncoder`. This demonstrates the model's applicability beyond the training/test split, for single-sample, real-world predictions.

9. Technology Stack

Component	Technology
Video I/O & Frame Processing	OpenCV (cv2)
Deep Learning Framework	TensorFlow / Keras
Numerical Computing	NumPy
Label Encoding, Splitting, Metrics	scikit-learn
Visualization	Matplotlib
Progress Tracking	tqdm

10. Project Structure

```
Activity_Recognition/  
■■■ Dataset/                # Video dataset (class-labeled folders)  
■■■ Activity Recognition Program.ipynb # Main notebook: full pipeline  
■■■ requirements.txt         # Python dependencies  
■■■ .gitignore              # Excludes venv, dataset, model weights  
■■■ README.md               # Project documentation
```

11. Limitations & Known Issues

- **Inference path bug:** the `predict_activity()` function ignores its own 'path' argument and always evaluates a hardcoded 'pth' variable defined outside the function — it must be corrected to use the passed-in parameter to correctly predict on arbitrary videos.
- **Input shape inconsistency:** the inference reshape produces a 4D tensor (1, 25, 24, 24), omitting the channel dimension used during training (25, 24, 24, 1). The current code executes without error, but the correct, robust reshape target is (1, 25, 24, 24, 1).
- **Low input resolution:** frames are downsized to 24×24 grayscale for computational efficiency; this may limit achievable accuracy as more visually similar activity classes are added.
- **Fixed frame sampling:** a uniform sampling interval may miss key motion moments in longer or variable-length videos.
- **Hardcoded file paths:** dataset and video paths are absolute Windows paths, reducing portability across environments.
- **Unused dependency:** `tensorflow_hub` is imported but not used anywhere in the current pipeline.
- **Legacy model format:** the trained model is saved in the HDF5 (.h5) format; TensorFlow recommends migrating to the native .keras format going forward.
- **Reproducibility:** no explicit random seed is set for frame sampling; only the train/test split uses a fixed seed.

12. Future Work

- Fix the two code-level issues identified in Section 11 (inference path bug and reshape dimension).
- Replace the custom CNN encoder with a pretrained backbone (e.g., MobileNet via TensorFlow Hub) for stronger spatial feature extraction via transfer learning.
- Increase frame resolution and/or the number of sampled frames per video if additional compute is available, and benchmark the resulting accuracy trade-off.
- Add a real-time webcam inference loop for live activity recognition demos.
- Convert the trained model to TFLite or ONNX for lightweight deployment on edge/mobile devices.
- Add a confusion matrix visualization alongside the classification report for deeper per-class error analysis.
- Migrate the saved model from the legacy HDF5 format to the native Keras (.keras) format.

13. Conclusion

This project demonstrates a complete, functional pipeline for video-based Human Activity Recognition, spanning raw video ingestion, preprocessing, model design, training, quantitative evaluation, and inference. By combining a TimeDistributed CNN for spatial feature extraction with an LSTM for temporal sequence modeling, the system achieved a strong **96.86% test accuracy** and a macro-averaged F1-score of 0.97 across seven distinct human activity classes. A small number of code-level issues were identified during review — most notably a path-handling bug in the inference function — and are documented above along with concrete recommendations for both correctness fixes and future capability improvements. Overall, the results validate the effectiveness of the CNN-LSTM approach for distinguishing activities that depend on motion, not just appearance.